MEDITECH_BUDDY: Project Documentation

1. Overview & purpose

MEDITECH_BUDDY (branded "Medi-Tech Buddy") is a student-style prototype of an AI-powered voice and vision healthcare assistant. The intended flow is: a user speaks their symptoms into a microphone, uploads a photo (for example a skin rash or acne), and receives a short doctor-like answer as text and as synthesized speech, all inside a Gradio web page.

The GitHub repository itself is minimal. It has one commit (21 May 2025, "Add files via upload", author Nikhil Rokade) and two tracked files: a one-line `README.md` (just the title) and a 1.5 MB archive named `AI POWERED HEALTHCARE ASSISTANT.zip`. All real content lives inside the zip, which was unpacked in a scratch location for this review. The repository was not modified.

The bundled README states the project is for educational purposes only and does not give real medical advice. The code is an early work in progress: the main entry point cannot run as written (see section 9).

2. Tech stack

Area	Technology
Language	Python 3
UI	Gradio (`gr.Interface`, microphone and image inputs)
Vision / language model	Google Gemini via `google-generativeai` (`gemini-1.5-flash`)
Speech to text	`SpeechRecognition` with the free Google Web Speech API
Text to speech	gTTS (Google) and ElevenLabs (`eleven_multilingual_v2`)
Audio conversion	`pydub` (needs ffmpeg)
Configuration	`python-dotenv` and a `.env` file
License	MIT (2025, "MEDITECH-BUDDY")

Note: the bundled PDFs describe an earlier design using Groq, Whisper and Llama 3 Vision. The code was later switched to Gemini, so the PDFs and the code disagree.

3. Repository structure

Tracked in git:

```
MEDITECH_BUDDY/
  README.md                              (one line: "# MEDITECH_BUDDY")
  AI POWERED HEALTHCARE ASSISTANT.zip    (1,535,235 bytes, the whole project)
```

Contents of the zip:

```
MEDITECH-BUDDY/
  .env  .gitignore  LICENSE  README.md  requirements.txt
  MEDITECH-BUDDY.pdf  MEDITECH-BUDDY_PRESENTATION.pdf   (identical size, 284,702 bytes)
  app/    meditech_app.py, __init__.py
  core/   brain_of_the_doctor.py, voice_of_the_doctor.py,
          voice_of_the_patient.py, __intit__.py
  utils/  config.py, __intit__.py
  assets/images/  acne.jpg, skin_rash.jpg
  assets/audio/input/   scalp-meditech.mp3, scalp.unknown, skinrash-meditech.mp3
  assets/audio/output/  elevenlabs_testing.wav, gTTS_testing.wav, patient_voice.wav
```

Observations: the package marker files in `core/` and `utils/` are misspelled `__intit__.py`, so those folders are not real regular packages (they still import as namespace packages). The `__init__.py` files are empty.

4. Architecture / workflow

Intended pipeline, as wired in `app/meditech_app.py`:

```
 Browser (Gradio page)
   |  mic recording (wav)        |  uploaded image
   v                             v
 process_inputs(audio, image)
```

```
   |
   |-- voice_of_the_patient.transcribe_with_gemini()
   |      SpeechRecognition.recognize_google  --> transcript
   |      Gemini 1.5 Flash: "interpret these symptoms" --> text  (returned as "speech to text")
   |
   |-- brain_of_the_doctor.analyze_image_with_query()
   |      prompt = system_prompt + that text, plus image bytes
   |      Gemini 1.5 Flash --> doctor_response (2-3 sentences)
   |
   |-- voice_of_the_doctor.text_to_speech_with_elevenlabs()
   |      writes assets/audio/output/final.wav, then plays it on the server
   v
 Gradio outputs: Speech-to-Text box, Response box, Audio player
```

Everything is a single synchronous function call per request. There is no database, no session state and no backend separate from the Gradio process.

5. Detailed walkthrough of the code

utils/config.py

Calls `load_dotenv()` and exposes `GEMINI_API_KEY` and `ELEVENLABS_API_KEY` from environment variables.

core/brain_of_the_doctor.py

- `encode_image(path)` reads the file bytes and returns a dict `{"mime_type": "image/jpeg", "data": bytes}`. The MIME type is hard-coded to JPEG regardless of the file.
- `analyze_image_with_query(query, image_payload)` creates a `gemini-1.5-flash` model and calls `generate_content([query, image])`. Any exception is returned as an error string beginning with a cross mark, so errors appear in the UI as if they were doctor answers.

core/voice_of_the_patient.py

- `record_audio(...)` records from the microphone with `speech_recognition`, adjusts for ambient noise, and saves a WAV. It is not used by the Gradio app (Gradio records audio itself). It calls `os.makedirs` but `os` is never imported, and `logging.basicConfig` is duplicated.
- `transcribe_with_gemini(stt_model, audio_filepath)` reads an audio file, transcribes it with `recognizer.recognize_google` (the `stt_model` argument is ignored), then sends the transcript to Gemini with the instruction "You are a medical assistant... Help interpret what it might be" and returns Gemini's reply. So the function does not return a transcript; it returns an AI interpretation. Errors are returned as strings.

core/voice_of_the_doctor.py

- `text_to_speech_with_gtts(text, wav_path)` creates an MP3 with gTTS, converts it to WAV with pydub, deletes the temp MP3, then plays the file.
- `text_to_speech_with_elevenlabs(text, wav_path, voice="Rachel", api_key=...)` calls `set_api_key`, `generate` and `save`. These names are never imported (only `ElevenLabs` from `elevenlabs.client` is), and they belong to the old ElevenLabs SDK, so this raises `NameError`. The exception is caught and printed, and the function returns without producing audio.
- `play_wav_audio(path)` plays the file on the server with PowerShell `SoundPlayer` (Windows), `afplay` (macOS) or `aplay` (Linux). On a hosted deployment this plays on the server, not the user's device.

app/meditech_app.py

Defines a long `system_prompt` instructing the model to act as a doctor "for learning purposes", answer in one paragraph of 2-3 sentences with no markdown, and start with "with what I see, I think you have...". `process_inputs` chains the three stages above and returns text, response and the output audio path. If no image is given it returns the fixed text "No image provided for me to analyse". The Gradio interface is created at import time and launched with `debug=True`, with no `if __name__ == "__main__"` guard.

Assets

Sample images (`acne.jpg`, `skin_rash.jpg`), sample patient recordings (`scalp-meditech.mp3`, `skinrash-meditech.mp3`, a duplicate `scalp.unknown`) and sample TTS outputs used for testing. The `.gitignore` lists the audio folders, but they were still zipped.

6. Data / configuration / dependencies

Environment variables expected in `.env`: `GEMINI_API_KEY` and `ELEVENLABS_API_KEY`.

`requirements.txt` (no version pins): `gradio`, `pydub`, `gtts`, `speechrecognition`, `google-generativeai`, `elevenlabs`, `python-dotenv`.

Missing or implicit requirements: `ffmpeg` on the PATH (pydub), `PyAudio` and PortAudio (microphone recording through `sr.Microphone`), and internet access to Google and ElevenLabs. The shipped `.env` uses `KEY: value` syntax rather than `KEY=value`, which python-dotenv may not parse as intended.

Data handled: user voice, user-provided medical images, symptom text. Nothing is stored beyond overwritten files under `assets/audio/`.

7. Setup and run instructions

The upstream README describes the following; the notes show what actually blocks it.

1. Clone the repo and unzip `AI POWERED HEALTHCARE ASSISTANT.zip`.
2. Create and activate a virtual environment: `python -m venv .medi-tech`.
3. Install dependencies: `pip install -r requirements.txt`, plus ffmpeg and PortAudio.
4. Create `.env` with your own keys: `GEMINI_API_KEY=...` and `ELEVENLABS_API_KEY=...`. Do not reuse the keys in the archive.
5. Run `python app/meditech_app.py` from the project root.

As written, step 5 fails. `meditech_app.py` imports `brain_of_the_doctor` and its siblings without the `core.` prefix, and the project root must be on `sys.path` for `utils.config`. Even after fixing imports, the call passes an unsupported `gemini_api_key` keyword to `transcribe_with_gemini`, which raises `TypeError`. A reasonable form after fixes is `python -m app.meditech_app` from the root, with `core.` imports corrected.

8. Security & privacy observations

This is the most important section for a medical-themed project.

- **Live API keys are committed.** The zip contains a populated `.env` with what appear to be real Gemini and ElevenLabs keys, and the repository is public on GitHub. The keys are intentionally not reproduced here. Treat them as compromised: revoke and rotate both, and purge them from git history if possible. The archive also bypasses the `.gitignore` that lists `.env`.
- **Literal key string.** `process_inputs` passes `api_key="ELEVENLABS_API_KEY"` (a quoted string), not the variable, which would fail authentication even if the SDK calls were right.
- **Health data leaves the machine.** Voice goes to the Google Web Speech API, symptom text and images go to Gemini, and the answer text goes to ElevenLabs. There is no consent notice, no retention policy and no anonymisation. If real patient data were used this would raise HIPAA, GDPR or similar concerns.
- **Safety of medical output.** The prompt tells the model to speak like a real doctor, avoid AI phrasing and give remedies, which can mislead users. The disclaimer appears only in the README, not in the UI. There is no triage or escalation path for emergencies.
- **Prompt injection.** Transcribed speech is concatenated directly into the prompt without separation.
- **Server-side playback and debug mode.** Audio plays on the host machine, and `debug=True` exposes tracebacks in the UI.

- **Subprocess use.** The PowerShell playback command interpolates a file path; paths are currently constants, so risk is low, but it is fragile.
- **No authentication and unpinned dependencies.** Anyone with the Gradio URL can spend the owner's API quota.
- **Personal data.** The bundled README lists personal email addresses of contributors.

9. Known limitations and suggested improvements

Functional defects found by reading the code:

1. Import errors in `meditech_app.py` (missing `core.` package prefix).
2. `transcribe_with_gemini` is called with a `gemini_api_key` argument it does not accept.
3. `os` is not imported in `voice_of_the_patient.py`.
4. ElevenLabs code uses removed functions (`set_api_key`, `generate`, `save`) that are not imported; the current SDK uses `ElevenLabs(api_key=...).text_to_speech.convert(...)`.
5. ElevenLabs returns MP3 audio but the file is saved as `.wav`.
6. The "Speech to Text" output is really a Gemini interpretation, and that interpretation is fed into the image prompt. This doubles the model calls and loses the patient's exact words.
7. If no image is given, the app gives up instead of answering from speech alone.
8. `gemini-1.5-flash` is retired; move to a current Gemini model. The MIME type should be detected, not forced to JPEG.
9. Sample MP3 inputs cannot be read by `sr.AudioFile`, which supports only WAV, AIFF and FLAC.
10. The gTTS function is unused, while the ElevenLabs function fails silently; there is no fallback.
11. Typos in the prompt ("wromg", "paragrap", "preambe") and in file names (`__intit__.py`).

Suggested improvements:

- Rotate the leaked keys, remove `.env` from the archive and history, and ship a `.env.example`.
- Unzip the project into real folders in the repository so it is reviewable, and write a proper top-level README. Remove the duplicate PDF and `scalp.unknown`.
- Pin dependency versions and document ffmpeg and PortAudio.
- Add a visible in-UI disclaimer, emergency guidance and refusal behaviour for serious symptoms.
- Return audio to the browser only (no server playback), add an `if __name__ == "__main__"` guard and disable `debug` outside development.
- Add tests, retry and timeout handling, and show errors in a separate field.
- Align documentation with the code (Gemini, not Groq/Llama) and consider a medically validated or fine-tuned model, as the PDF itself suggests, plus multilingual support.